

Milestone 3

Team z-f

Game: Uno

Team Members: Jonathan Mai, Ivan Wu, Megha Rai

Game overview

The game will be the classic game of Uno.

Players will be able to join or leave a room.

In the room players will be able to:

- Play cards
- Draw cards
- Call Uno

The server will manage all the logic, like decks, hands, discard, turn order, and validation of moves

API Endpoints

Game Management Endpoints

- Create game
 - POST /api/games/
- Join game
 - POST /api/games/:game_id/join
- Start game
 - POST /api/games/:game_id/start
- Get game state
 - GET /api/games/:game_id
- Leave game
 - POST /api/games/:game_id/leave

Game Action Endpoints

- Draw card
 - POST /api/games/:game_id/draw
- Play card
 - POST /api/games/:game_id/cards/:card_id/play
- Call Uno
 - POST /api/games/:game_id/uno

Endpoint Deep Dive #1: Play Card

HTTP Method & Route: POST /api/games/:game_id/cards/:card_id/play

Validations:

- Game with game_id exists
- User is in the game_user table for this game
- The current game user matches the game_user.id
- game.state === "inMatch"
- The card exists in a player's hands ie. card_pile === 'Hand'
- The card is owned_by === game_user.id
- If not first turn, then
 - The card's color value matches the top of the discard pile or
 - The card's number value matches the top of the discard pile or
 - The card is a Wild/WildDrawFour
 - The card's chosen color is a valid color

Endpoint Deep Dive #1: Play Card - Continued

State Updates:

- Move `game_cards.id` from hand to discard
 - `game_cards.id = game_card_id`
 - `card_pile = 'Discard'`
 - `owned_by = NULL`
- Update `game_user.hand_count -= 1`
- Update the top discard card with the played card

Events Triggered:

- `game:state:update` → update the card piles for all players
- `game:hand:update` → For the player who played the card
- `game:ended` → Only if a player wins will this activate
- `game:card:played`

Endpoint Deep Dive #2: Join game

- HTTP Method & Route: POST /api/games/:game_id/join
- Authorization:
 - Must be logged in
 - The game must be visible according to the room privacy
- Validation Checks:
 - A game with game_id exists
 - `game.status === open`
 - The number of players in game does not exceed maxplayers
 - The joining user is not already a game_user

Endpoint Deep Dive #2: Join game - Continued

- State Updates:
 - Insert a row into the game_user table
 - game_id
 - user_id
 - hand_count = 0
 - seat_at = next lowest int
- Socket.io Events
 - game:player:joined → All players in the room

Socket.io events

- `game:player:joined` - A user has joined the game
- `game:player:left` - A user has left the game
- `game:state:update` - When the state of the game change
- `game:hand:update` - (private) When a player's hands change
- `game:ended` - When the game has ended
- `game:turn:changed` - Turn changes from current to next player
- `game:card:played` - A player successfully played a card
- `game:card:drawn` - (private) player drew a card